TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 8

Pydantic request bodies, routers, dependencies, containerizing

Phase 2 — Building Real Apps · Weeks 7–12 · Modules 03–04

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. 03.05 Request Bodies with Pydantic
2. 03.06 Multiple Routes and Routers
3. 03.07 Dependencies and Injection
4. 03.08 Containerizing FastAPI

Learning objective: Validate JSON request bodies on a `POST /items` route with a Pydantic `BaseModel` and test valid and invalid payloads from both curl and Swagger UI.

Lesson 05: Request Bodies with Pydantic

By the end, you will have a `POST /items` route that accepts a JSON body, validated against a Pydantic model, and you will have hit it from both curl and Swagger UI.

~30 minutes of typing

> **Pydantic** is the data-validation library FastAPI sits on top of. You define a class with typed fields; FastAPI uses it to parse, validate, and document request bodies.

---

1. Make sure your venv is active and uvicorn is running with `--reload`.

```
cd ~/docker-course/fastapi-app && source .venv/bin/activate
```

2. Open `main.py`.

3. Add the Pydantic import. Below `from fastapi import FastAPI`, add:

```
from pydantic import BaseModel
```

4. Save. Watch uvicorn reload — no errors expected.

5. Below the imports, define an `Item` model. Type it carefully:

```


class Item(BaseModel):
    name: str
    price: float
    is_offer: bool | None = None
```

6. Save. Confirm the model is part of the file.

```
grep -A 4 "class Item" main.py
```

You should see the four lines you just typed.

7. Add a `POST /items` route below your existing routes.

```
@app.post("/items")
def create_item(item: Item):
    return item
```

8. Save. Watch uvicorn reload.

9. Curl the new route with a valid JSON body. **Note the three flags:** `-X POST`, `-H "Content-Type: application/json"`, `-d '{...}'`.

```
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget", "price": 9.99}'
```

```
{"name":"Widget","price":9.99,"is_offer":null}
```

10. Curl with an offer flag.

```
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget", "price": 9.99, "is_offer": true}'
```

```
{"name":"Widget","price":9.99,"is_offer":true}
```

11. Curl with an invalid body — missing `price`.

```
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'
```

```
{"detail":[{"type":"missing","loc":["body","price"],"msg":"Field required",...}]}
```

Pydantic rejected it with a 422. Notice the response tells you exactly which field is missing.

12. Confirm the status.

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget"}'
```

```
422
```

13. Curl with a wrong type — `price` as a string.

```
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget", "price": "free"}'
```

```
{"detail":[{"type":"float_parsing","loc":["body","price"],"msg":"Input should be a
valid number,...","input":"free"}]}
```

14. Forget the `Content-Type` header on purpose.

```
curl -X POST http://localhost:8000/items \
  -d '{"name": "Widget", "price": 9.99}'
```

```
{"detail":[{"type":"json_invalid",...}]}
```

(FastAPI couldn't parse the body. **Always set `Content-Type: application/json` on POST.**)

15. Now hit it from Swagger UI. Open `http://localhost:8000/docs` and refresh the page.

16. You should see `POST /items` listed. Click it → **"Try it out"**.

17. The request body box has a sample JSON pre-filled. Change it to:

```
{
  "name": "From Swagger",
  "price": 1.23,
  "is_offer": false
}
```

18. Click **Execute**. The response should echo your input.

19. In the same Swagger box, delete `"price": 1.23,` and click Execute. Swagger shows the 422 inline with the same error structure curl saw.

20. Now let's actually store items. Above the routes, add a tiny in-memory list:

```
fake_items_db: list[Item] = []
```

21. Update the POST handler to save the item:

```
@app.post("/items")
def create_item(item: Item):
    fake_items_db.append(item)
    return item
```

22. Add a route to list all items:

```
@app.get("/items")
def list_items():
    return fake_items_db
```

23. Save.

24. POST a couple of items.

```
curl -X POST http://localhost:8000/items -H "Content-Type: application/json" -d
'{"name": "Apple", "price": 0.50}'
curl -X POST http://localhost:8000/items -H "Content-Type: application/json" -d
'{"name": "Bread", "price": 2.50, "is_offer": true}'
```

25. List them.

```
curl http://localhost:8000/items
```

```
[{"name":"Apple","price":0.5,"is_offer":null},
{"name":"Bread","price":2.5,"is_offer":true}]
```

26. Restart the server (Ctrl+C in the uvicorn terminal, then `uvicorn main:app --reload` again). List items again.

```
curl http://localhost:8000/items
```

```
[]
```

(In-memory list — gone after restart. We'll fix this with a real DB in lesson 10.)

27. Log it.

```
echo "POST /items + Pydantic validation: working" >> ~/docker-course/notes/notes.txt
```

Watch out

- `Content-Type: application/json` on every POST. Forgetting it is the #1 cause of unexplained 422 errors. Drill it.
- The single quotes around the `-d '{...}'` payload matter — they prevent your shell from interpreting `$` or `"` inside the JSON.
- `is_offer: bool | None = None` — the default value is what makes the field optional. Without `= None`, Pydantic would require it.
- The in-memory `fake_items_db` is **not** real persistence. Restart the server and it's gone. We'll fix that in lesson 10.

Checkpoint

```
curl -s -X POST http://localhost:8000/items -H "Content-Type: application/json" -d '{"name": "T", "price": 1}'
```

You should see the item echoed back as JSON.

```
curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8000/items -H "Content-Type: application/json" -d '{"name": "T"}'
```

You should see `422`.

```
curl -s http://localhost:8000/items | python -m json.tool
```

You should see at least one item from the previous POST formatted as JSON.

In the browser at `/docs`, the `POST /items` row should show your `Item` schema with `name`, `price`, `is_offer`.

Next up: 06-multiple-routes-and-routers.md — Split the app into separate router files

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Refactor routes into per-resource modules using `APIRouter` with prefixes and tags, and mount them in `main.py` with `app.include_router`.

Lesson 06: Multiple Routes and Routers

By the end, you will have refactored the items routes into a separate `routers/items.py` module, added a `routers/users.py`, and mounted both in `main.py` with `APIRouter`.

~25 minutes of typing

> One file per resource. As your app grows, you don't want every route in `main.py` — you want each resource (items, users, orders, …) in its own file. `APIRouter` is FastAPI's mechanism for that.

---

1. Make sure your venv is active and uvicorn is running with `--reload`.

```
cd ~/docker-course/fastapi-app && source .venv/bin/activate
```

2. Create the routers directory.

```
mkdir routers
```

3. Make it a Python package by adding an empty `__init__.py`.

```
touch routers/__init__.py
```

4. Confirm.

```
ls routers/
```

```
__init__.py
```

5. Create `routers/items.py`.

```
touch routers/items.py
```

6. Open `routers/items.py` in VS Code.

7. Type the imports:

```
from fastapi import APIRouter
from pydantic import BaseModel
```

8. Create the router with a prefix and a tag (the tag is what shows up as the section header in Swagger).

```


router = APIRouter(prefix="/items", tags=["items"])
```

9. Move the `Item` model from `main.py` to here:

```


class Item(BaseModel):
    name: str
    price: float
    is_offer: bool | None = None
```

10. Add the in-memory store:

```


fake_items_db: list[Item] = []
```

11. Add the routes — note `@router.get`, not `@app.get`. And the prefix `/items` is already on the router, so paths here are relative.

```
@router.get("")
def list_items():
    return fake_items_db


@router.get("/{item_id}")
def read_item(item_id: int, q: str | None = None, short: bool = False):
    if short:
        return {"item_id": item_id}
    return {"item_id": item_id, "q": q}


@router.post("")
def create_item(item: Item):
    fake_items_db.append(item)
    return item
```

12. Save.

13. Now create the users router.

```
touch routers/users.py
```

14. Open `routers/users.py` and type:

```
from fastapi import APIRouter
from pydantic import BaseModel


router = APIRouter(prefix="/users", tags=["users"])


class User(BaseModel):
    name: str
    email: str


fake_users_db: list[User] = []


@router.get("")
def list_users():
    return fake_users_db


@router.post("")
def create_user(user: User):
    fake_users_db.append(user)
    return user
```

15. Save.

16. Now slim down `main.py` to just import and mount the routers. Open `main.py` and replace its contents with:

```
from fastapi import FastAPI

from routers import items, users

app = FastAPI()


@app.get("/")
def read_root():
    return {"hello": "fastapi"}


app.include_router(items.router)
app.include_router(users.router)
```

17. Save. Watch uvicorn reload. If you see `ImportError`, check that `routers/__init__.py` exists.

18. Confirm the project structure.

```
ls -la
ls -la routers/
```

You should see:

```
.
├── .gitignore
├── .venv/
├── main.py
├── requirements.txt
└── routers/
    ├── __init__.py
    ├── items.py
    └── users.py
```

19. Curl the moved items route. The path is the same as before because the router has prefix `/items`.

```
curl http://localhost:8000/items
```

```
[]
```

20. Curl the new users route.

```
curl http://localhost:8000/users
```

```
[]
```

21. POST a user.

```
curl -X POST http://localhost:8000/users \
  -H "Content-Type: application/json" \
  -d '{"name": "Ada", "email": "ada@example.com"}'
```

```
{"name":"Ada","email":"ada@example.com"}
```

22. List users to confirm it stuck.

```
curl http://localhost:8000/users
```

```
[{"name":"Ada","email":"ada@example.com"}]
```

23. POST an item to confirm the items router still works.

```
curl -X POST http://localhost:8000/items \
  -H "Content-Type: application/json" \
  -d '{"name": "Widget", "price": 5.0}'
```

24. Hit the path-param route on items.

```
curl http://localhost:8000/items/42
```

```
{"item_id":42,"q":null}
```

25. Open `/docs` in the browser. Refresh. You should now see **two sections** ("items" and "users") thanks to the `tags=` argument on each router.

26. Click the **"items"** tag header in Swagger — the items routes collapse/expand together. Same for "users".

27. Hit `POST /users` from Swagger UI to confirm it works there too.

28. Log it.

```
echo "Routers: items + users split out" >> ~/docker-course/notes/notes.txt
```

Watch out

- **`__init__.py` in `routers/`** — without it, `from routers import items` fails with `ModuleNotFoundError`.
- The `prefix="/items"` is already on the router, so don't repeat it: write `@router.get("")` for `/items`, not `@router.get("/items")` (which would become `/items/items`).
- `app.include_router(items.router)` — note the `.router`. The router *object* is what FastAPI mounts, not the module.
- When you move code between files, the in-memory state moves too. The `fake_items_db` you POSTed to in lesson 05 is gone — it lived in the old `main.py`. The new one is empty until you POST again.

Checkpoint

```
curl -s http://localhost:8000/items
```

Should return `[]` or whatever you POSTed since the refactor.

```
curl -s http://localhost:8000/users
```

Should return `[]` or your test users.

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/items/42
```

Should return `200`.

```
ls routers/
```

Should show `__init__.py`, `items.py`, `users.py`.

In `/docs` you should see **two tagged sections**: "items" and "users".

Next up: 07-dependencies-and-injection.md — Use `Depends` to share logic across routes

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Implement a header-checking `get_current_user` dependency with `Depends` and apply it at route level and router level, distinguishing 401 and 422 responses.

Lesson 07: Dependencies and Injection

By the end, you will have a `get_current_user` dependency that reads a header, and a protected route that returns 401 without the header and 200 with it.

~25 minutes of typing

> A **dependency** in FastAPI is a function whose return value is "injected" into your route. Common uses: auth, DB sessions, shared validation. The pattern is `param = Depends(my_function)`.

---

1. Make sure venv is active and uvicorn is running.

```
cd ~/docker-course/fastapi-app && source .venv/bin/activate
```

2. Create a `dependencies.py` at the project root.

```
touch dependencies.py
```

3. Open it in VS Code.

4. Type the imports:

```
from fastapi import Header, HTTPException
```

5. Add the dependency function. It reads an `X-Token` header and rejects bad ones.

```


def get_current_user(x_token: str = Header(...)):
    if x_token != "secret-token":
        raise HTTPException(status_code=401, detail="Invalid token")
    return {"username": "admin"}
```

6. Save. Confirm.

```
cat dependencies.py
```

7. Now use it on a route. Open `routers/items.py`.

8. Add to the imports at the top:

```
from fastapi import APIRouter, Depends

from dependencies import get_current_user
```

(Replace the existing `from fastapi import APIRouter` line — combine into one.)

9. Add a new protected route at the bottom of `routers/items.py`.

```


@router.get("/me/protected")
def protected(user: dict = Depends(get_current_user)):
    return {"message": f"Hello {user['username']}, you reached a protected route"}
```

10. Save. Watch uvicorn reload.

11. Curl without the header.

```
curl -i http://localhost:8000/items/me/protected
```

```
HTTP/1.1 422 Unprocessable Entity
...
{"detail":[{"type":"missing","loc":["header","x-token"],"msg":"Field
required",...}]}
```

(Pydantic 422 because the header is required at the framework level.)

12. Curl with the wrong token.

```
curl -i http://localhost:8000/items/me/protected -H "X-Token: nope"
```

```
HTTP/1.1 401 Unauthorized
...
{"detail":"Invalid token"}
```

13. Curl with the right token.

```
curl -i http://localhost:8000/items/me/protected -H "X-Token: secret-token"
```

```
HTTP/1.1 200 OK
...
{"message":"Hello admin, you reached a protected route"}
```

14. Now apply the dependency at the **router level** to protect every users route. Open `routers/users.py`.

15. Update the imports:

```
from fastapi import APIRouter, Depends

from dependencies import get_current_user
```

16. Update the router instantiation to include the dependency:

```
router = APIRouter(
    prefix="/users",
    tags=["users"],
    dependencies=[Depends(get_current_user)],
)
```

17. Save.

18. Curl `/users` without the header.

```
curl -i http://localhost:8000/users
```

```
HTTP/1.1 422 Unprocessable Entity
```

19. Curl `/users` with the right token.

```
curl -i http://localhost:8000/users -H "X-Token: secret-token"
```

```
HTTP/1.1 200 OK
```

20. Curl `/users` with the wrong token.

```
curl -i http://localhost:8000/users -H "X-Token: nope"
```

```
HTTP/1.1 401 Unauthorized
```

21. Curl `/items` (the items router has **no** router-level dependency).

```
curl -i http://localhost:8000/items
```

```
HTTP/1.1 200 OK
```

(Items is unprotected — only the `/items/me/protected` route uses Depends.)

22. Open `/docs` in the browser. Refresh.

23. Click any user route → **"Try it out"**. Notice Swagger now shows an `X-Token` header field. Type `secret-token`, click Execute.

24. Try with a wrong value to see the 401 in Swagger.

25. Use the global "Authorize" button — for our simple X-Token, you set the token via the header field on each route. (Real apps use OAuth/Bearer tokens with `OAuth2PasswordBearer`; we're keeping it simple.)

26. Why this matters. A dependency is a regular function. You can swap it (e.g., a fake `get_current_user` for tests). You can compose them (`Depends` inside another `Depends`). This is how FastAPI handles auth, DB sessions, rate limits, feature flags — all the same pattern.

27. Log it.

```
echo "Dependencies + injection: get_current_user wired in" >> ~/docker-course/notes/notes.txt
```

Watch out

- The header name in `Header()` is automatically converted between hyphens and underscores. `x_token: str = Header(...)` matches `X-Token` on the wire.
- `Header(...)` (with the literal `...` Ellipsis) means "required." `Header(None)` means "optional, default None."
- `Depends(get_current_user)` — pass the function, **don't call it**. No parentheses.
- Router-level dependencies apply to **every route on that router**. If you only want to protect one, put `Depends` on the route, not the router.

Checkpoint

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/items/me/protected
```

Should print `422` (header missing).

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/items/me/protected -
H "X-Token: nope"
```

Should print `401`.

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/items/me/protected -
H "X-Token: secret-token"
```

Should print `200`.

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/users
```

Should print `422` (header missing — the users router is now protected).

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/items
```

Should print `200` (items list is still unprotected).

Next up: 08-containerizing-fastapi.md — Wrap the whole app in a Dockerfile

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Write a Dockerfile that caches the pip-install layer, build the image, and run the containerized app with uvicorn bound to `0.0.0.0` so the mapped port is reachable.

Lesson 08: Containerizing FastAPI

By the end, you will have a Dockerfile that builds your FastAPI app into an image, and you will be able to run that image with `docker run` and hit it with curl.

~30 minutes of typing

> The Module 02 muscle memory pays off here. You're going to write a Dockerfile from scratch, build it, and run it. Same `docker run` patterns you drilled before.

---

1. Make sure you're in the project directory. **Stop the running uvicorn** with Ctrl+C in its terminal — we're about to use port 8000 from a container instead.

```
cd ~/docker-course/fastapi-app
```

2. Create an empty `Dockerfile`.

```
touch Dockerfile
```

3. Open `Dockerfile` in VS Code.

4. Type the base image line. We use the `slim` variant of Python 3.12 — small but enough.

```
FROM python:3.12-slim
```

5. Set the working directory inside the container.

```
WORKDIR /app
```

6. Copy `requirements.txt` first (before the rest of the code). This is the **layer-caching trick**: if requirements don't change, Docker reuses the pip-install layer between builds.

```
COPY requirements.txt .
```

7. Install the dependencies inside the image.

```
RUN pip install --no-cache-dir -r requirements.txt
```

8. Now copy the rest of the code.

```
COPY . .
```

9. Expose the port the app will listen on (this is documentation more than networking — `-p` on `docker run` is what actually publishes it).

```
EXPOSE 8000
```

10. Set the command to run when the container starts. **Note** `--host 0.0.0.0` — without that, uvicorn only listens on the container's loopback and `docker run -p` won't reach it.

```
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

11. Save. Confirm the file.

```
cat Dockerfile
```

You should see seven lines (the seven directives above).

12. Create a `.dockerignore` to keep your venv and other junk out of the image.

```
touch .dockerignore
```

13. Open `.dockerignore` and add:

```
.venv/
__pycache__/
*.pyc
.git/
.gitignore
Dockerfile
.dockerignore
```

14. Save.

15. Build the image. The `.` at the end is the **build context** — the current directory.

```
docker build -t fastapi-app .
```

You should see a series of step lines (`[1/6] FROM python:3.12-slim`, `[2/6] WORKDIR /app`, etc.), then `Successfully tagged fastapi-app:latest`.

16. Confirm the image exists.

```
docker images | grep fastapi-app
```

```
fastapi-app   latest   abc123...   30 seconds ago   180MB
```

17. Run the container. `-d` for detached, `-p` to map host port 8000 to container port 8000, `--name` for friendly identification.

```
docker run -d -p 8000:8000 --name fastapi-app fastapi-app
```

```
abc123def456...
```

18. Confirm it's running.

```
docker ps --filter name=fastapi-app
```

You should see status `Up`.

19. Hit it.

```
curl http://localhost:8000
```

```
{"hello":"fastapi"}
```

20. Hit the items route.

```
curl http://localhost:8000/items
```

```
[]
```

(Fresh container — empty list.)

21. Hit the protected route without the header.

```
curl -s -o /dev/null -w "%{http_code}\n" http://localhost:8000/items/me/protected
```

```
422
```

22. Hit it with the right header.

```
curl http://localhost:8000/items/me/protected -H "X-Token: secret-token"
```

```
{"message":"Hello admin, you reached a protected route"}
```

23. Open `http://localhost:8000/docs` in the browser. Same Swagger UI, now served from the container.

24. Look at the container's logs.

```
docker logs fastapi-app
```

You should see the uvicorn startup banner and entries for each request you've made.

25. Make a code change to confirm the container is **isolated** from your local files. In VS Code, change `main.py`'s `read_root` return to `{"hello": "from-host"}`. Save.

26. Curl the container.

```
curl http://localhost:8000
```

```
{"hello":"fastapi"}
```

(Unchanged — the image was built with the old code. The container doesn't see your edits.)

27. Rebuild the image.

```
docker build -t fastapi-app .
```

28. Restart the container with the new image.

```
docker stop fastapi-app && docker rm fastapi-app
docker run -d -p 8000:8000 --name fastapi-app fastapi-app
```

29. Curl again.

```
curl http://localhost:8000
```

```
{"hello":"from-host"}
```

(Now the new code is in the image.)

30. Revert your change in `main.py` back to `{"hello": "fastapi"}` and save (we don't need to rebuild — we'll use Compose with bind mounts in the next lesson for hot reload inside Docker).

31. Stop and remove the container — we'll let Compose own the lifecycle from now on.

```
docker stop fastapi-app && docker rm fastapi-app
```

32. Log it.

```
echo "Dockerfile + image build + container run: working" >> ~/docker-course/notes/notes.txt
```

Watch out

- `--host 0.0.0.0` in `CMD`. Without it, uvicorn binds only to the container's loopback and `curl http://localhost:8000` from the host returns "connection reset." This is the most common Dockerfile mistake.
- The `.dockerignore` file matters — without it, your `.venv/` (~150 MB) gets copied into the image, ballooning it. Keep `.venv/` out.
- `COPY requirements.txt .` **before** `COPY . .`. If you copy everything first, every code change invalidates the pip-install cache and re-installs every package on every build. That's slow.
- Code changes don't appear in the container until you **rebuild and restart**. That's why we use Compose with bind mounts next — for dev parity with the venv hot-reload.

Checkpoint

```
docker images | grep fastapi-app
```

Should show your image.

```
docker build -t fastapi-app .
```

Should complete without errors.

```
docker run --rm -d -p 8000:8000 --name test-fastapi fastapi-app && sleep 2 && curl -s http://localhost:8000 && docker stop test-fastapi
```

Should print `{"hello":"fastapi"}` and stop the container.

Next up: 09-docker-compose-and-postgres.md — Add a Postgres database and orchestrate both with Compose

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.